Chair of Methods for Model-based Development in Computational Engineering | RWTH AACHEN UNIVERSITY

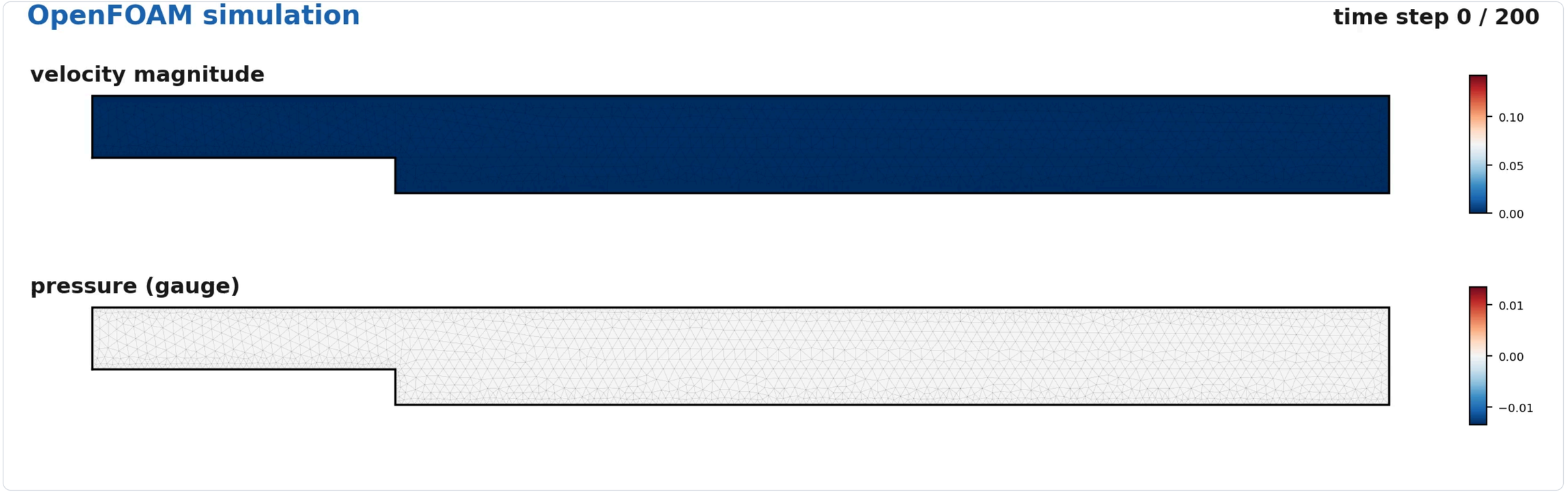
MASTER'S THESIS DEFENSE

Hybrid Simulations with GNN Preconditioning for Flow Simulations

Abhishek Nain

Supervisor Ingo Steldermann · Examiner Prof. Julia Kowalski

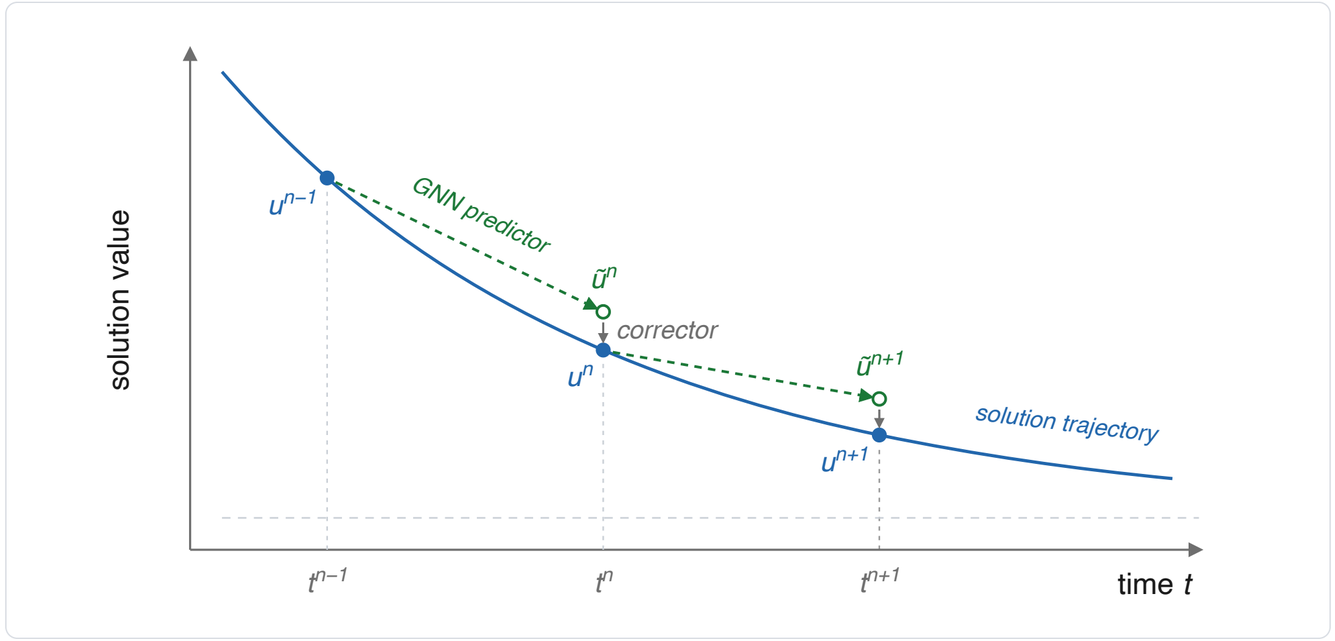
CFD is powerful, but expensive



Every time step solves a large sparse system to convergence, and that **iterative solver work** dominates the cost of a high-fidelity run.

ACT 1 · THE IDEA

A learned predictor, an OpenFOAM corrector



↻

GNN predictor
proposes a field close to the solution

predicted field

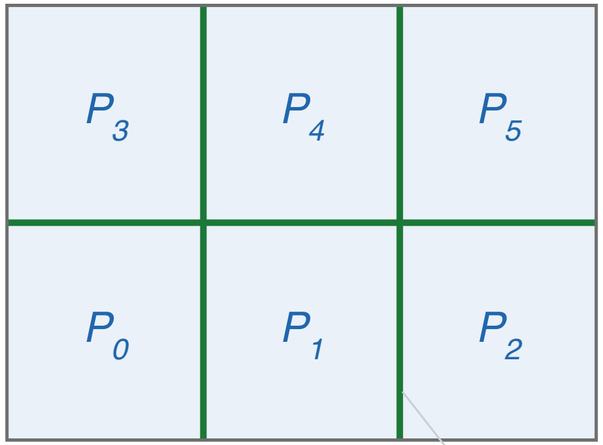
OpenFOAM corrector
iterates the solver to tolerance

Converged solution
exact, guaranteed

Each step is solved iteratively to convergence; the GNN predicts a field close to the solution, so OpenFOAM reaches tolerance in **fewer iterations** and still **guarantees the answer**.

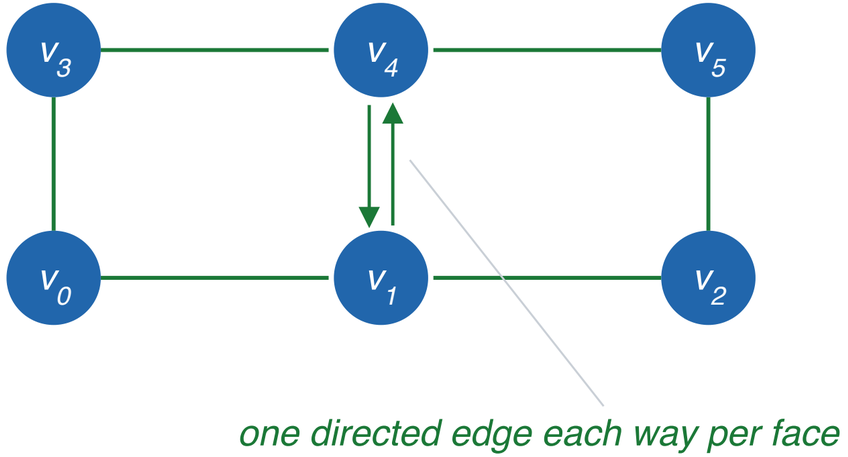
The mesh is already a graph the solver assembles

(a) Finite-volume mesh



cells couple across their shared faces

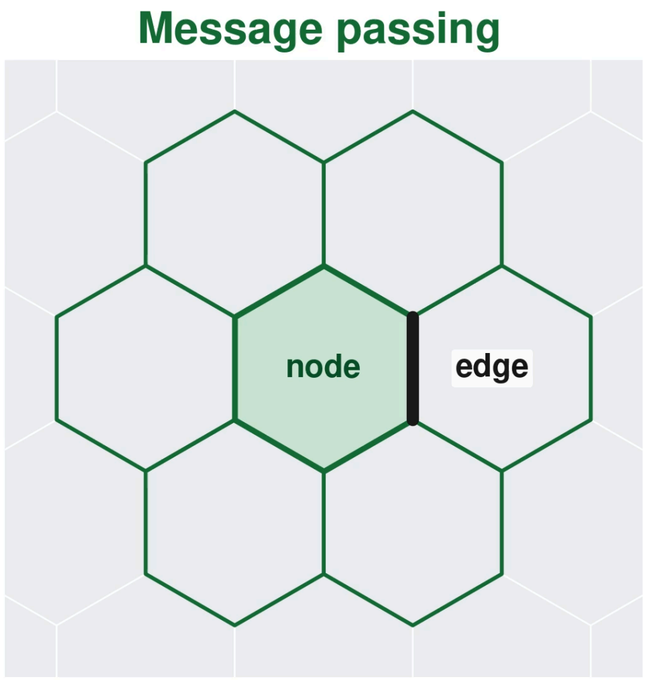
(b) Graph representation



cells P_i → nodes v_i · internal faces → *edges*

Cells become **nodes**, internal faces become **edges**. No grid, no resampling.

Message passing is local finite-volume coupling



one cell and its neighbours

G = (V, E)

nodes V are cells, edges E are shared faces

1 · update each edge

$$e'_{ij} = e_{ij} + \mathrm{MLP}([\, e_{ij},\ h_i,\ h_j \,])$$

2 · sum the messages

$$m_i = \sum_j e'_{ij}$$

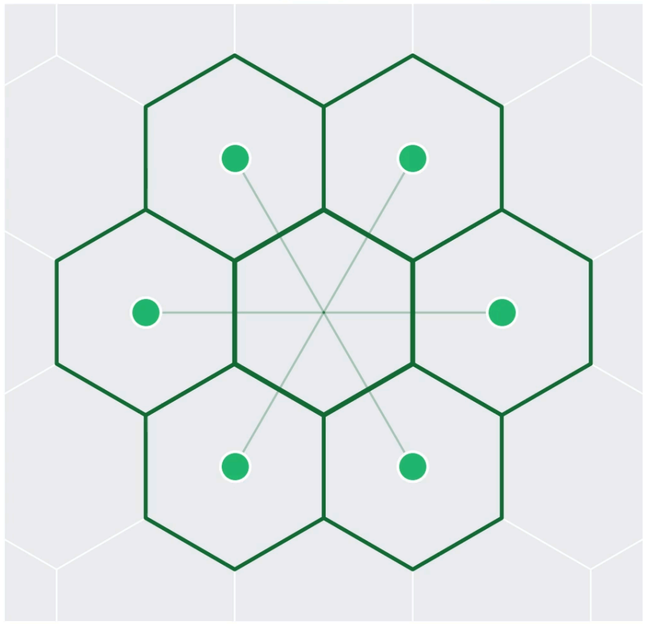
3 · update each node

$$h'_i = h_i + \mathrm{MLP}([\, h_i,\ m_i \,])$$

A cell and its **face-neighbours**.

Message passing is local finite-volume coupling

Message passing



a message glides along each shared edge into the cell

G = (V, E)

nodes V are cells, edges E are shared faces

1 · update each edge

$$e'_{ij} = e_{ij} + \mathrm{MLP}([\, e_{ij},\ h_i,\ h_j \,])$$

2 · sum the messages

$$m_i = \sum_j e'_{ij}$$

3 · update each node

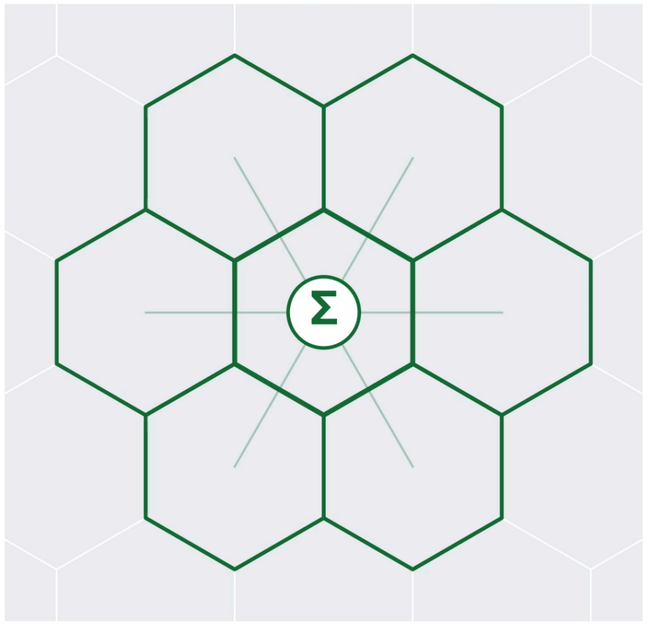
$$h'_i = h_i + \mathrm{MLP}([\, h_i,\ m_i \,])$$

h node vector e edge vector
m summed message

Edge update: combine each edge with its two cells.

Message passing is local finite-volume coupling

Message passing



the cell sums the incoming messages

G = (V, E)

nodes V are cells, edges E are shared faces

1 · update each edge

$$e'_{ij} = e_{ij} + \mathrm{MLP}([\, e_{ij},\ h_i,\ h_j \,])$$

2 · sum the messages

$$m_i = \sum_j e'_{ij}$$

3 · update each node

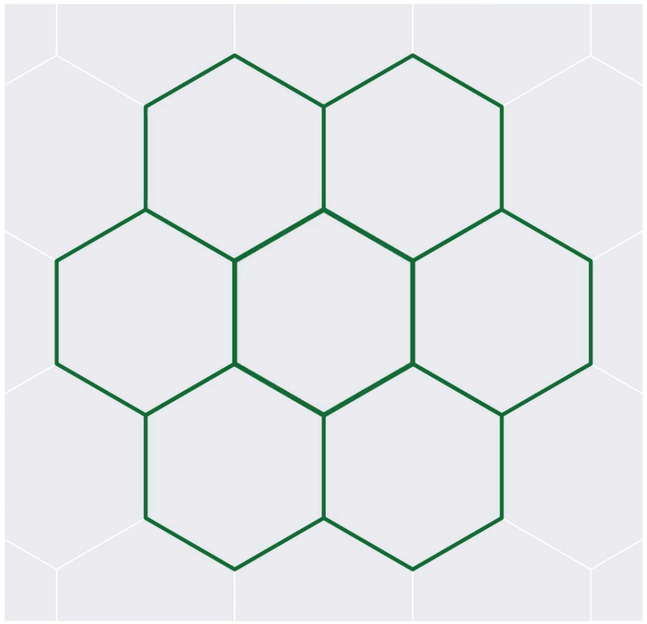
$$h'_i = h_i + \mathrm{MLP}([\, h_i,\ m_i \,])$$

h node vector e edge vector
m summed message

Sum the messages, like a flux balance.

Message passing is local finite-volume coupling

Message passing



and updates its own state

G = (V, E)

nodes V are cells, edges E are shared faces

1 · update each edge

$$e'_{ij} = e_{ij} + \mathrm{MLP}([\, e_{ij},\ h_i,\ h_j \,])$$

2 · sum the messages

$$m_i = \sum_j e'_{ij}$$

3 · update each node

$$h'_i = h_i + \mathrm{MLP}([\, h_i,\ m_i \,])$$

h node vector e edge vector
m summed message

Node update, with a skip connection.

Message passing is local finite-volume coupling

Message passing

now zoom out: every cell does the same, every round

G = (V, E)

nodes V are cells, edges E are shared faces

1 · update each edge

$$e'_{ij} = e_{ij} + \mathrm{MLP}([\, e_{ij},\ h_i,\ h_j \,])$$

2 · sum the messages

$$m_i = \sum_j e'_{ij}$$

3 · update each node

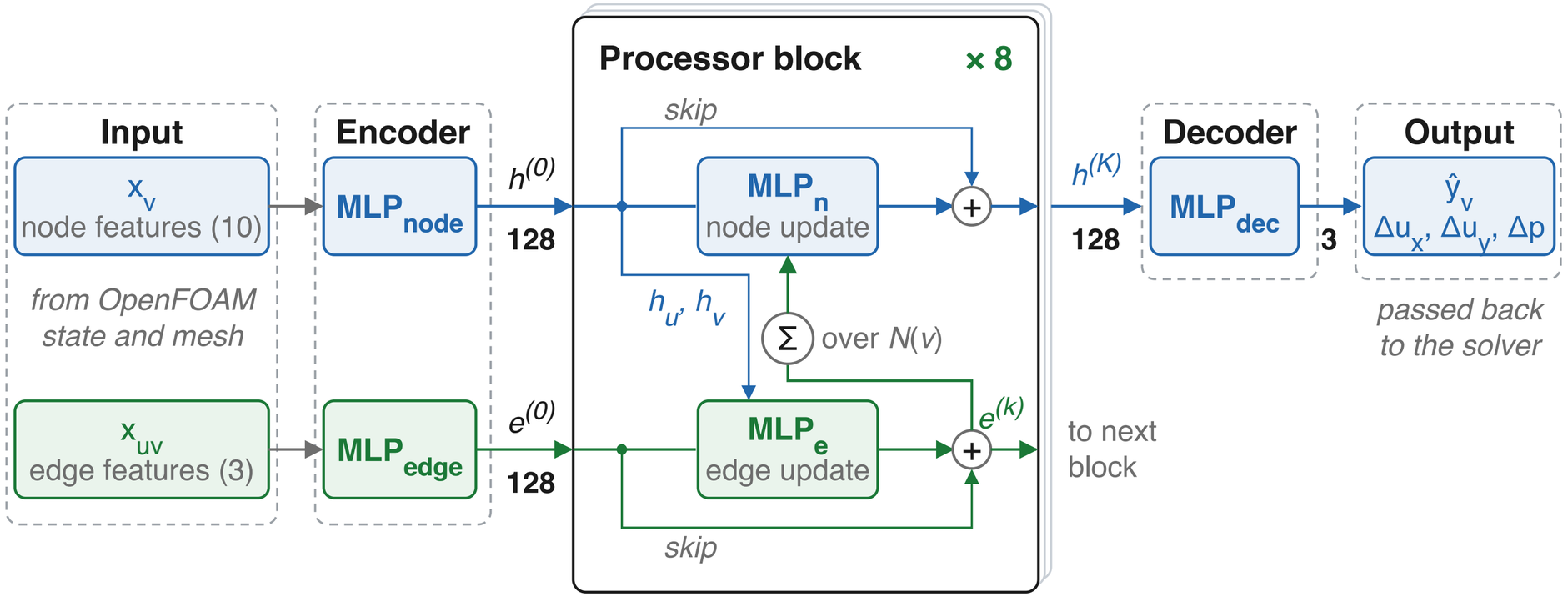
$$h'_i = h_i + \mathrm{MLP}([\, h_i,\ m_i \,])$$

× 8 rounds

8-hop receptive field

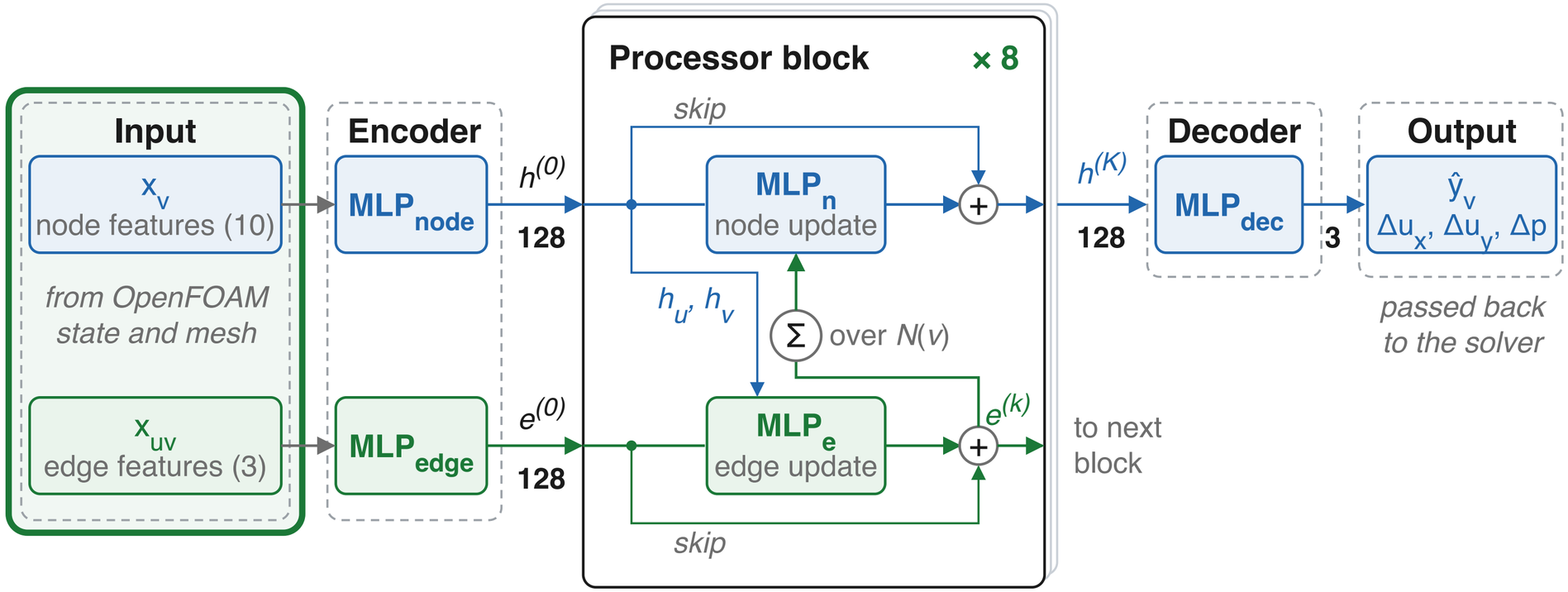
Eight rounds reach an **eight-hop receptive field**.

Predict a field increment, not the full solution



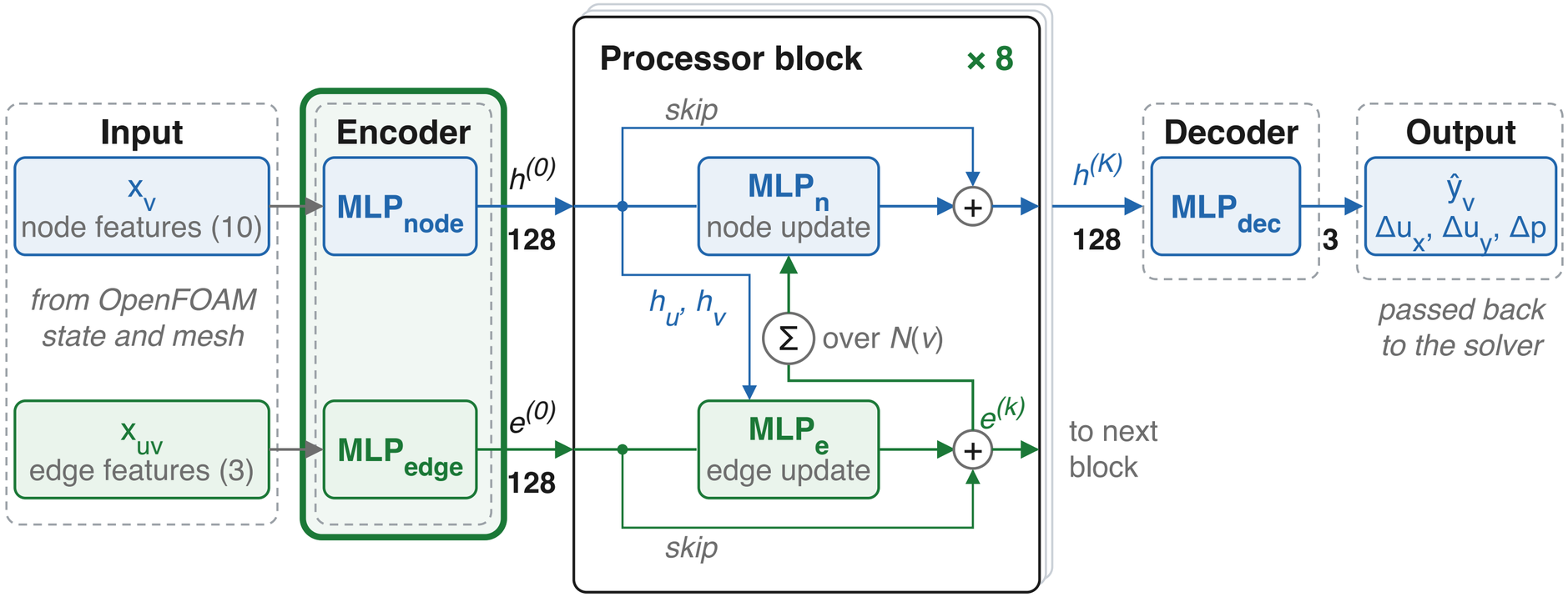
Encoder · processor ×8 · decoder on the mesh graph.

Predict a field increment, not the full solution



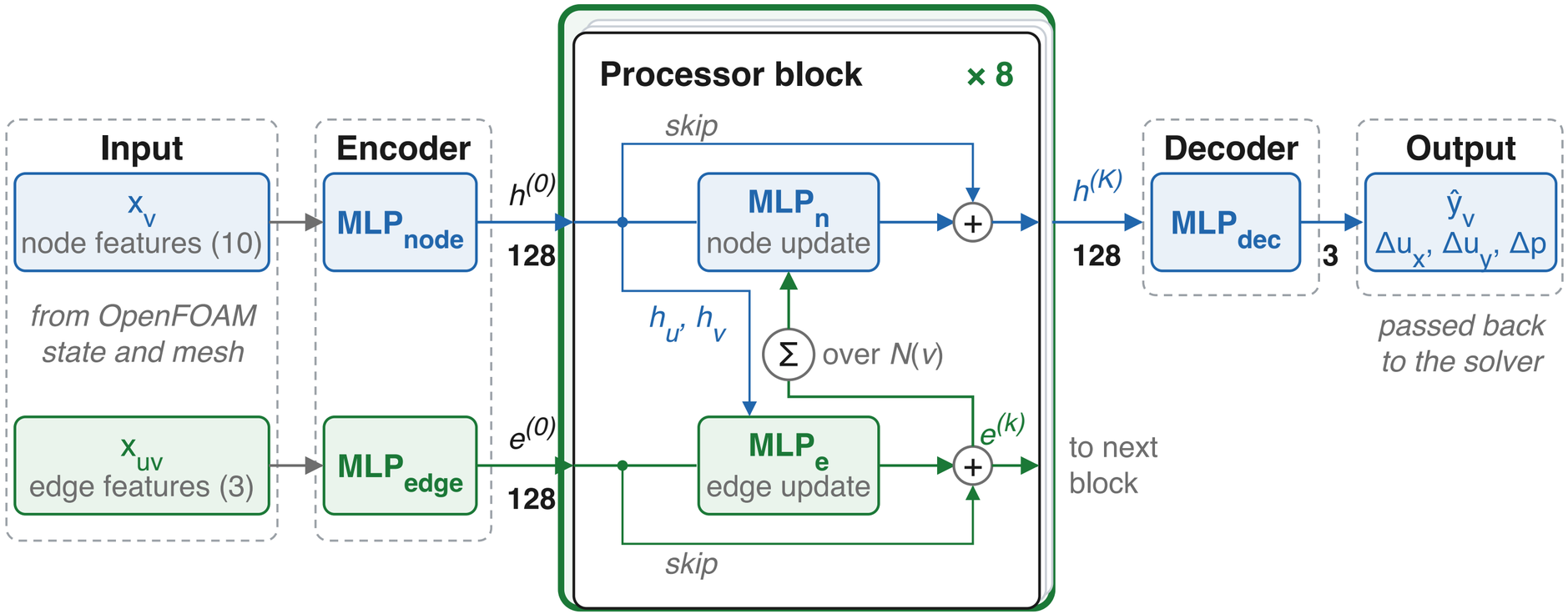
Input: 10 node features, 3 edge features.

Predict a field increment, not the full solution



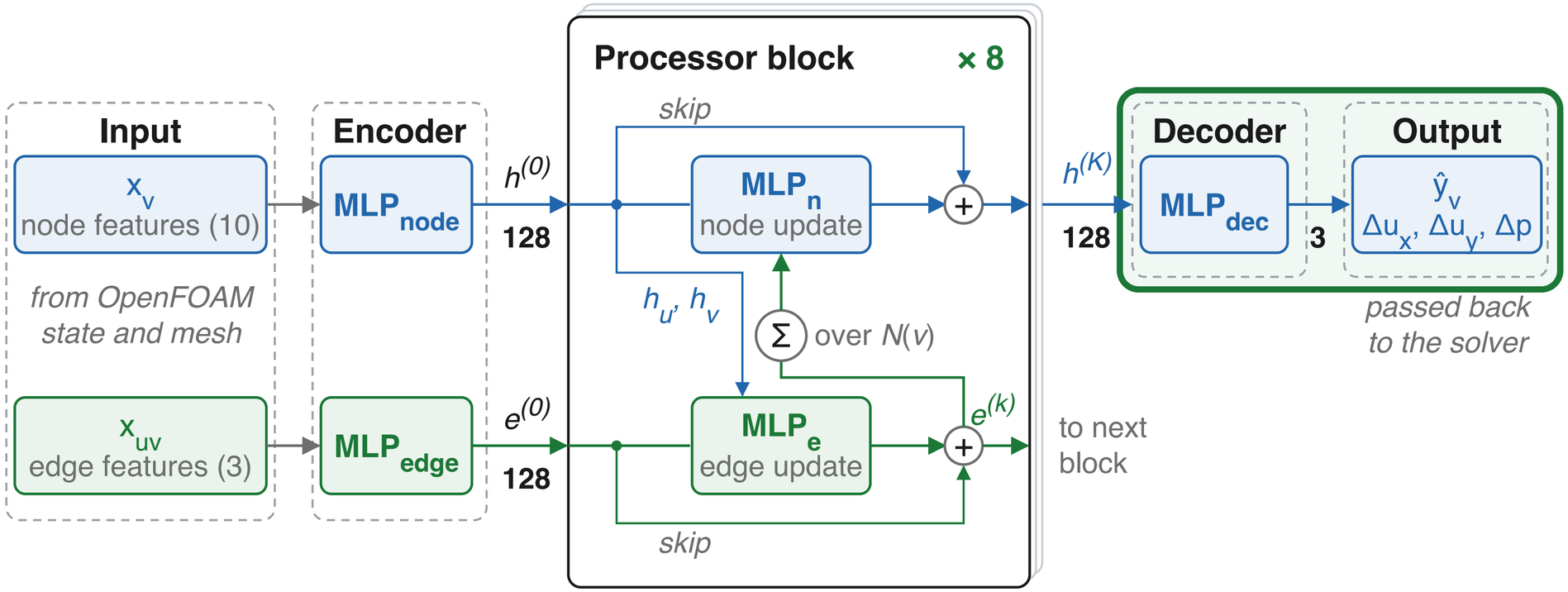
Encoder lifts them to a 128-d latent.

Predict a field increment, not the full solution



Processor ×8: edge update, sum, node update.

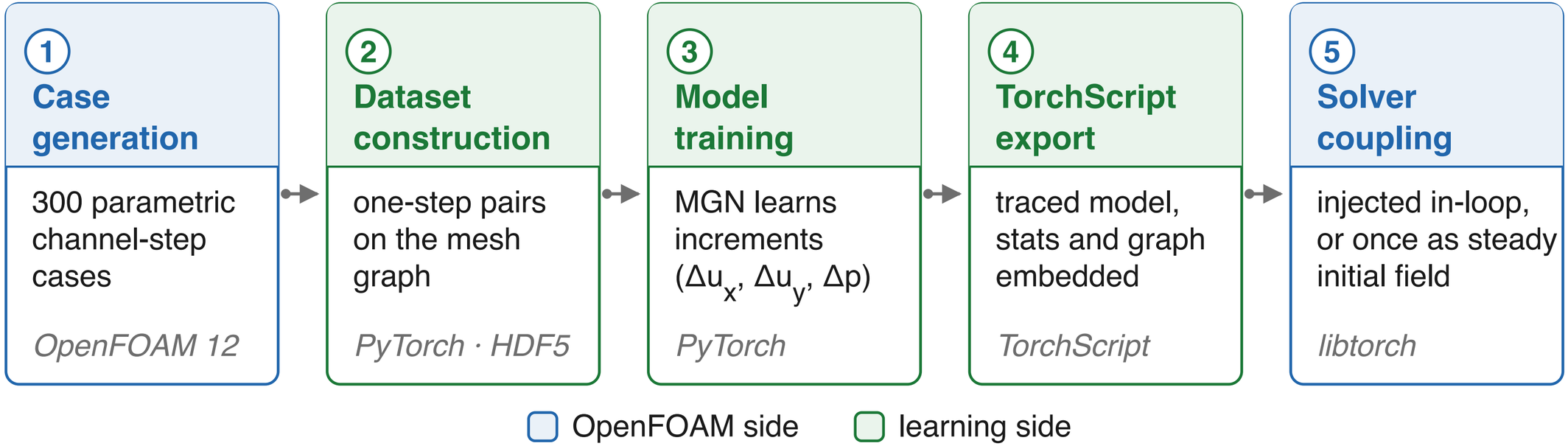
Predict a field increment, not the full solution



Decoder reads out a per-cell **increment**.

ACT 2 · OVERVIEW

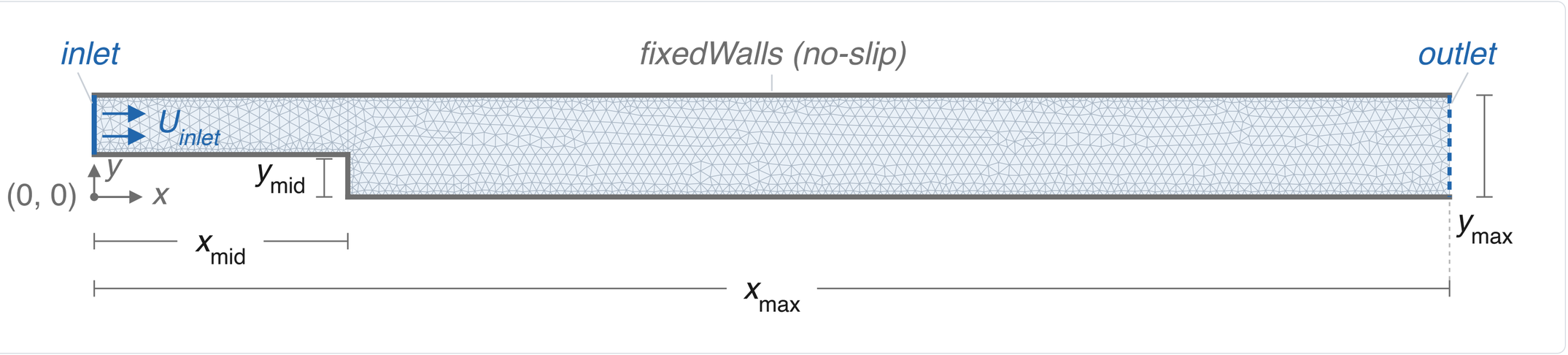
The pipeline, end to end



Generate data, train the model offline, then **couple it to OpenFOAM** at run time.

ACT 2 · DATA

300 geometries, one model, 30 held out

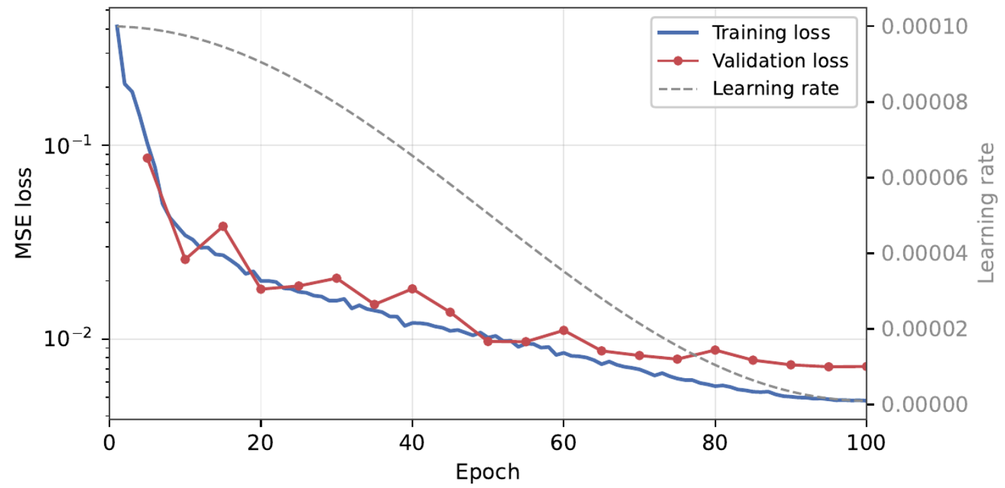


Inlet velocity U	Step height H	Step length L
0.10–0.40 m/s	**8–15** mm	**60–100** mm

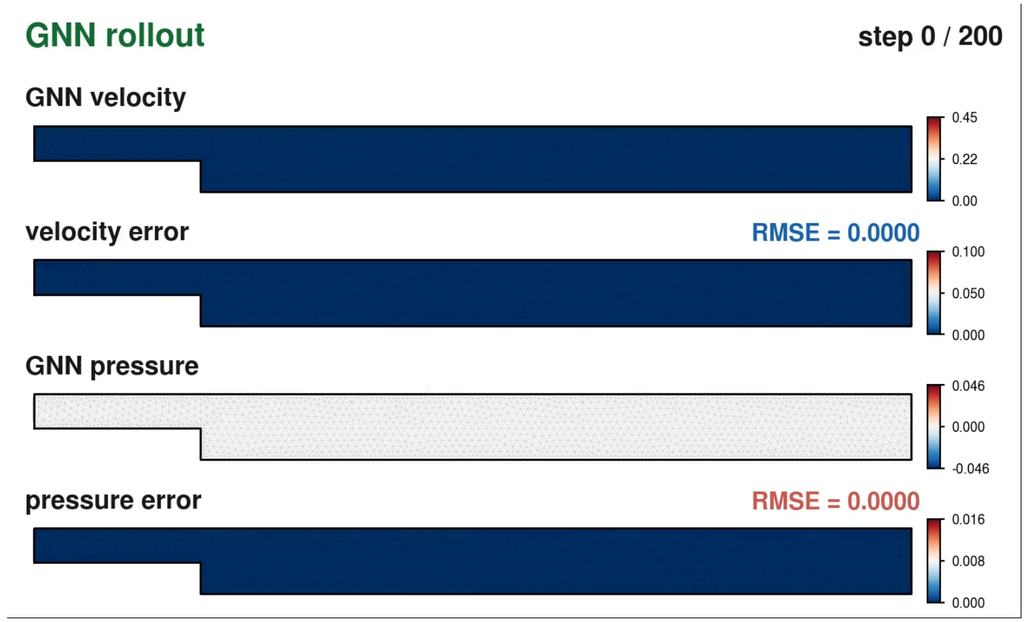
Parametric channel-with-step family; train on **240**, test on **30 unseen cases**.

ACT 2 · TRAINING

Trained one step ahead; the rollout tracks the solver



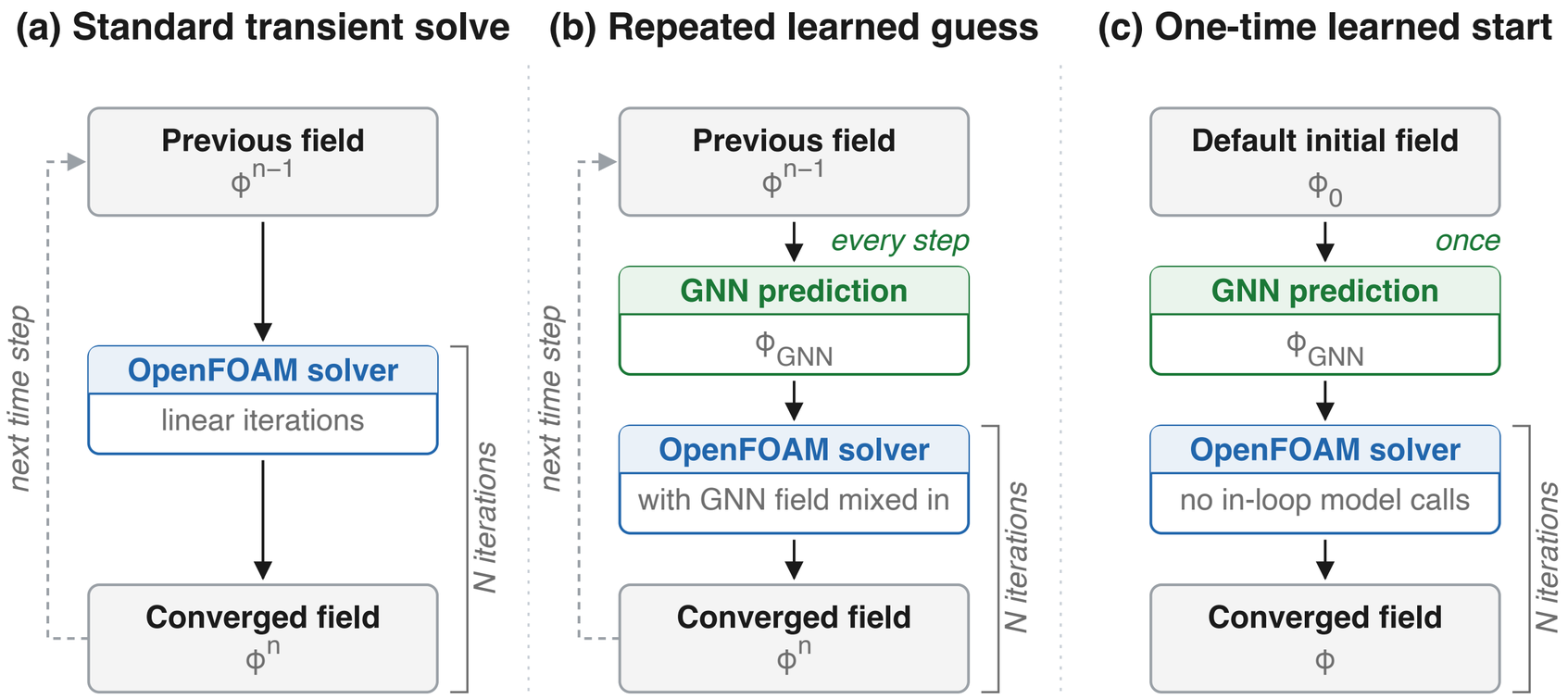
Training and validation loss



GNN rollout vs OpenFOAM, 200 steps

Trained only on one-step next-field error; rolled out 200 steps the learned fields stay close to OpenFOAM, with the **error growing slowly**.

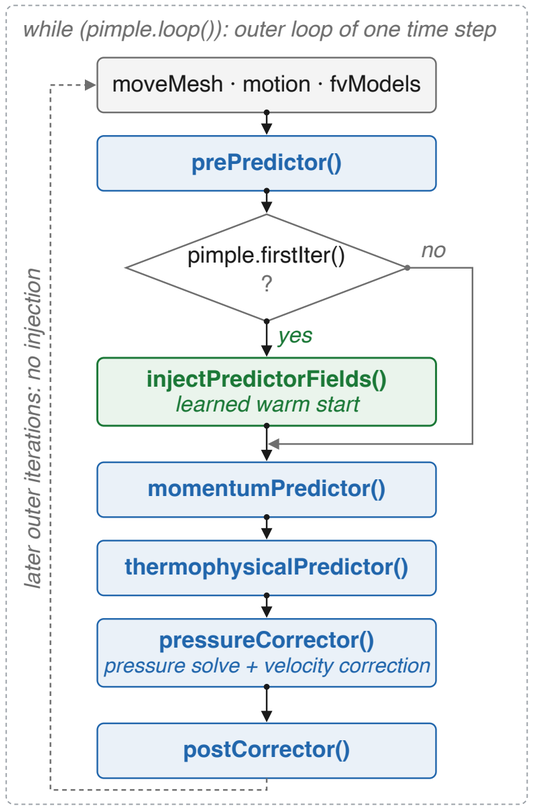
Two ways to apply the predictor-corrector



Inject a fresh prediction at **every transient step**, or write one developed rollout **once** to start a steady solve.

ACT 2 · INTEGRATION

One hook in PIMPLE



Inject · blend

$$\Phi_{inject} = (1-\alpha)\,\phi_{current} + \alpha\,\Phi_{predicted}$$

α · scale to the timestep

$$\alpha = \Delta t_{OpenFOAM} / \Delta t_{GNN}$$

the model's one-step increment is trained at $\Delta t_{GNN} = 0.01$, and α rescales it to the solver step.

Two settings we run

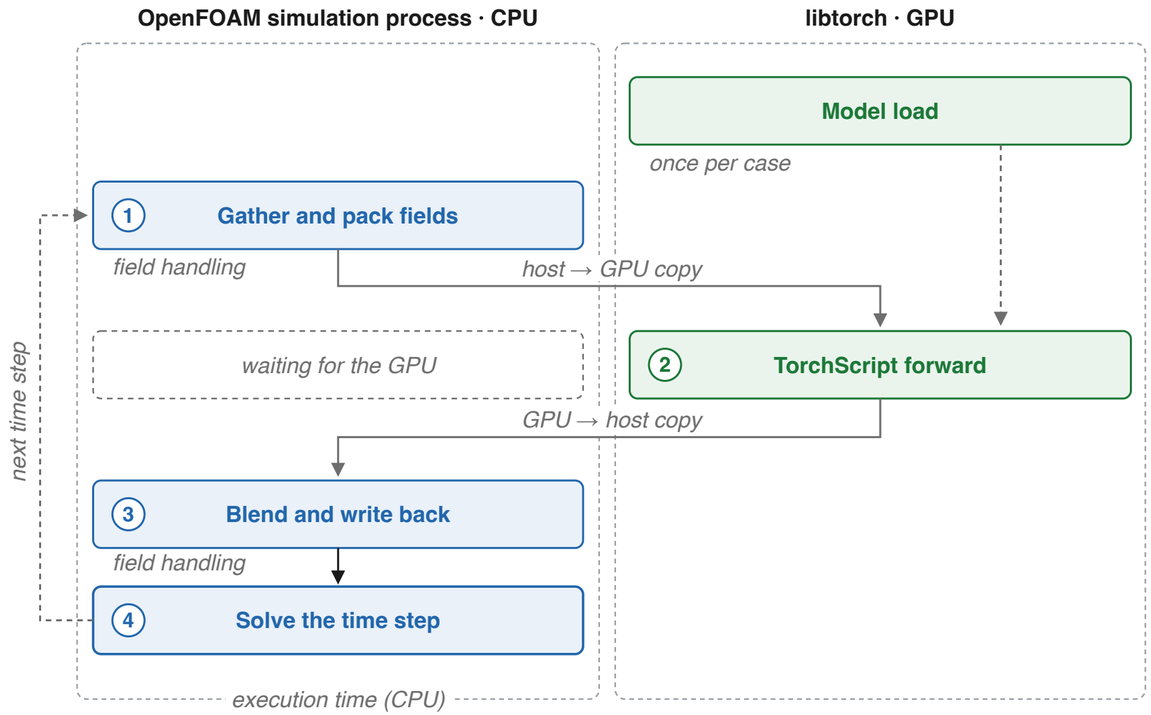
Matched $\Delta t_{OF} = 0.01 = \Delta t_{GNN} \rightarrow \alpha = 1$

Fine-step $\Delta t_{OF} = 0.002 \rightarrow \alpha = 0.2$

Injected once, at the **first outer PIMPLE iteration**, before the pressure solve.

ACT 2 · INTEGRATION

Anatomy of one injection



Gather and pack, GPU forward, blend and write back, solve; the model **loads once per case**. That data movement is **field handling**.

ACT 3 · VALIDATION

The coupling itself is neutral

Sanity controls · iteration change vs the cold solver

Control	Total	Pressure	What it shows
`identityNoop`	**0.00%**	**0.00%**	*reproduces the solver exactly*
`identity_model`	**0.00%**	**0.00%**	*exact match through the model path*
`bad_zero`	**+13.58%**	**+12.52%**	*a deliberately bad field adds work*

Per-injection overhead, isolated by the identity no-op · + model load ~0.9 s once per case

Stage	Identity no-op	Real GNN run
`GNN forward`	**0.14 ms**	**3.7–10.8 ms**
`Field handling`	**~5.0 ms**	**~5.5 ms**

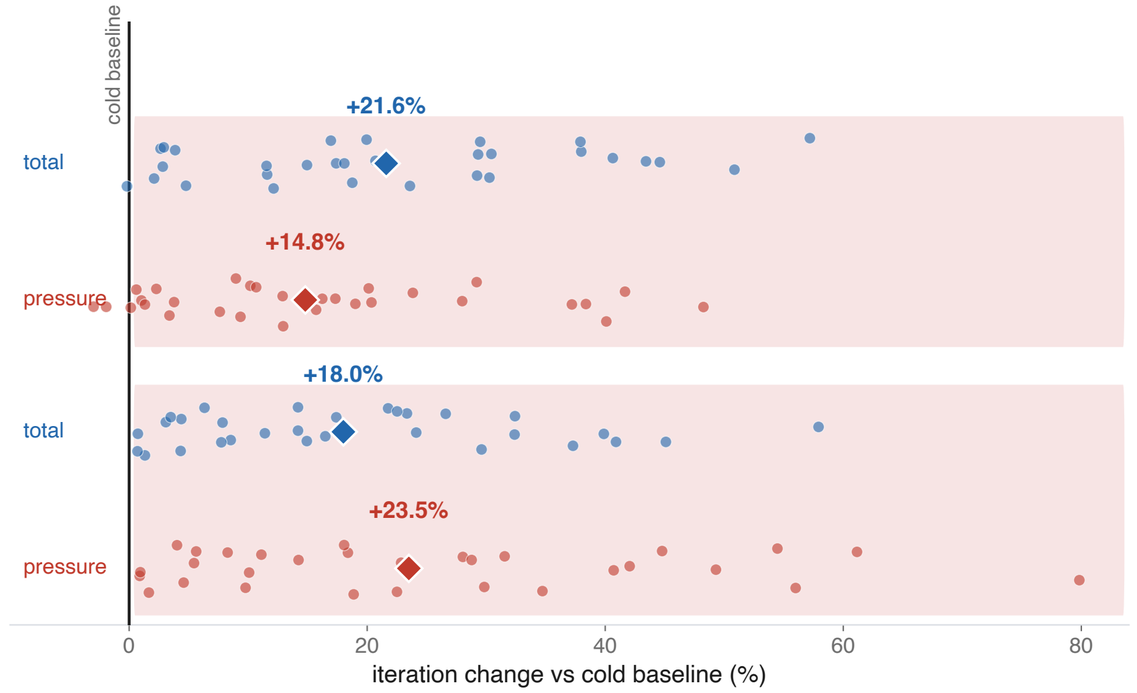
On its own the coupling adds no iterations, only field quality does. The cost is **per-injection overhead**, field handling and the forward, and it **scales with how often you inject**.

ACT 3 · RESULT A

Injecting every step makes it worse

Matched
Δt = 0.01, α = 1
1 of 30 improved

Fine-step
Δt = 0.002, α = 0.2
none improved

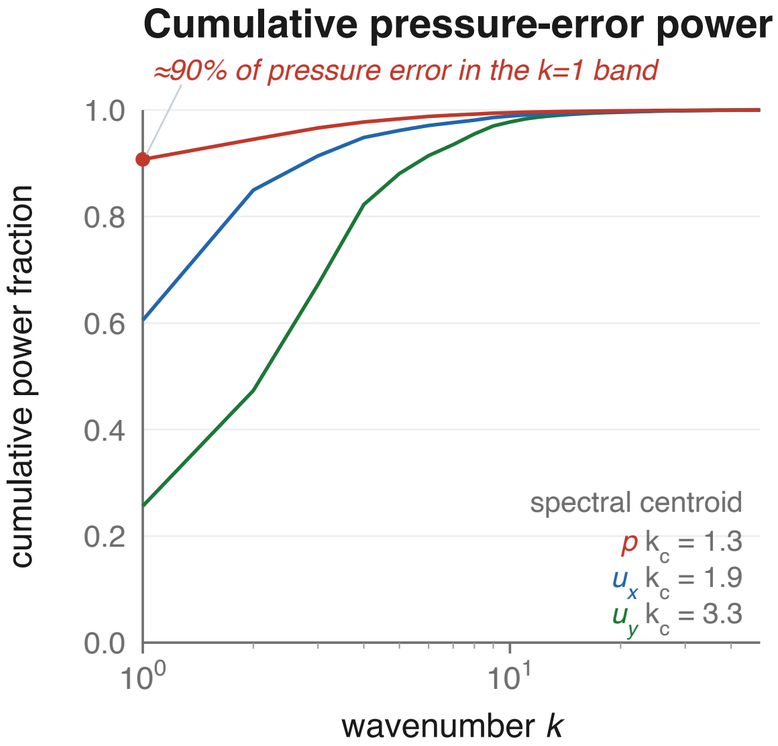


Repeated transient injection: **+21.6%** iterations matched, **+18.0%** fine-step; improves 1 of 30.

under-reaching detail

ACT 3 · WHY

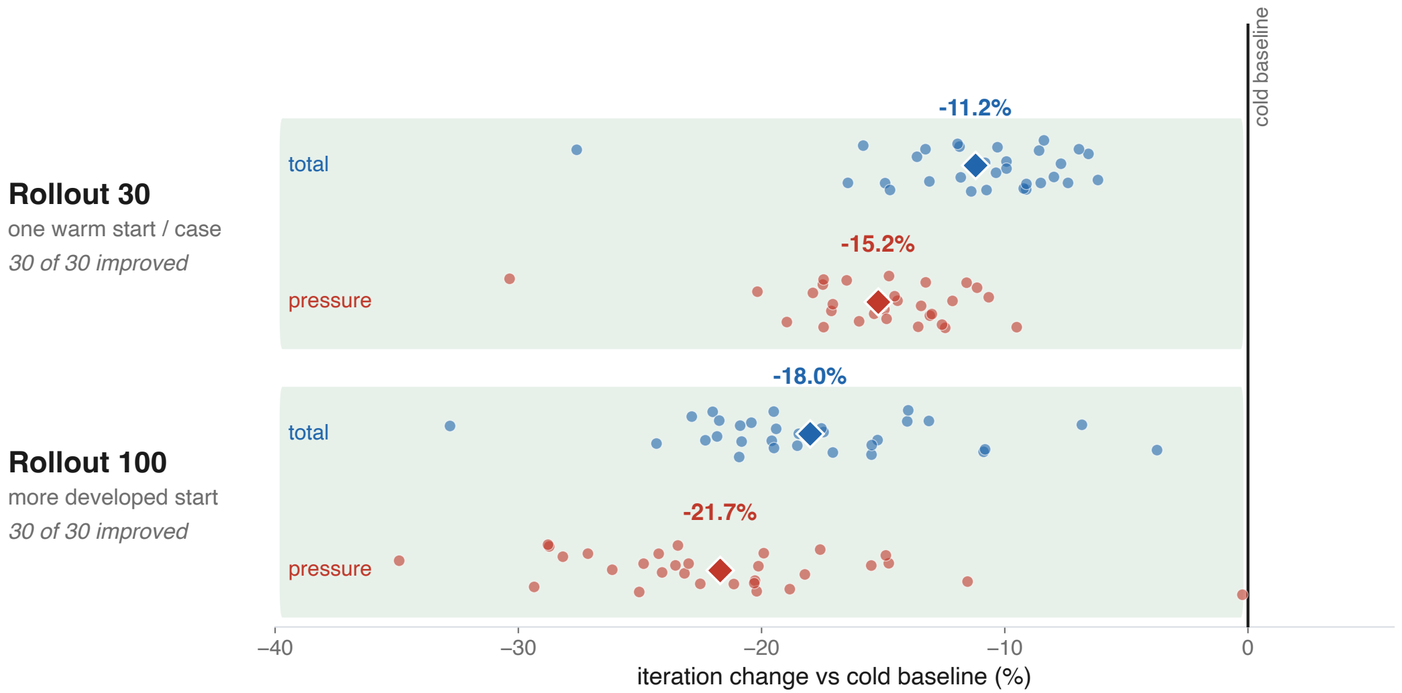
The pressure error lives at the global scale



About **90%** of the pressure error is in the lowest-wavenumber band, exactly what a fixed-depth GNN cannot reach in one pass.

ACT 3 · RESULT B

As a one-time steady warm start, it works in every case



Steady initialization: **11% to 18%** fewer total iterations, **15% to 22%** fewer pressure iterations, all 30 cases improve.

full cost sheet

ACT 3 · COST

Iteration savings, not yet a wall-clock speed-up

Learned-side overhead, measured

Stage	Wall-clock	How often
`Model load`	**1.07 s**	*once per case*
`GNN forward`	**10.9 ms**	*per call*
`Steady warm-start rollout`	**0.13–0.27 s**	*once per case*
`Transient field handling`	**~5.5 ms**	*per step*

An **iteration reduction**, not yet an end-to-end wall-clock speed-up. The forward and field handling add real time per injection, so the cost **scales with injection count**, once per case for steady, every step for transient.

What this thesis shows

1. A learned warm start **reduces steady-solver iterations in every test case**.
2. Repeated transient injection **does not help** this one-step model.
3. The deliverable is a **reusable OpenFOAM coupling interface**.

Where it goes next

1. Train for **solver response**, not field error (unrolled and differentiable-solver templates now exist).
2. Learned **preconditioners** as a complementary route.
3. Harder cases: **higher Reynolds, tighter tolerances, 3D**.

MBD Chair of Methods for Model-based Development in Computational Engineering | RWTH AACHEN UNIVERSITY

THANK YOU

Questions?

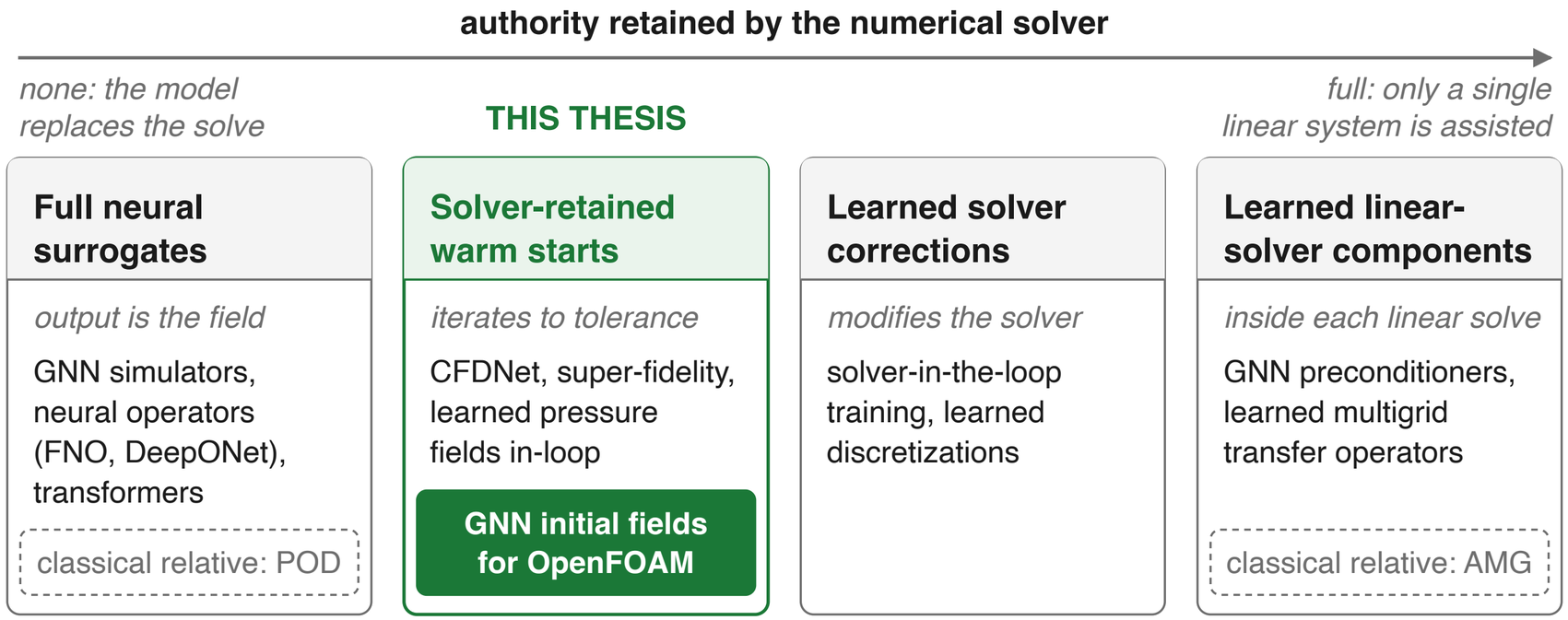
APPENDIX

Backup slides

Detail and source material held in reserve for questions.

BACKUP · B1

Where this sits among ML-CFD approaches



Between a **full surrogate** and **learned solver-internal components**: solver-retained, a learned initial guess.

back to talk

BACKUP · B2

Full configuration and architecture study

Mainline checkpoint · architecture and training

Hyperparameter	Value
Hidden dimension	**128**
Message-passing steps	**8**
MLP hidden layers	**2**
Aggregation / norm	**sum / LayerNorm**
Node / edge / output dim	**10 / 3 / 3**
Trainable parameters	**≈1.29 M**
Optimizer	**Adam, $10^{-4} \rightarrow 10^{-6}$ cosine**
Epochs / batch	**100 / 32**

Pilot architecture study · 48 training cases

Config (hidden / steps)	Pilot val loss
96 / 8	**0.05631**
96 / 12	**0.05146**
128 / 8 *selected*	**0.03433**
128 / 12 *lowest*	**0.02464**

128 / 8 *chosen over the lower-loss 128 / 12 for cheaper inference at every injection. Full* **300-case** *training (240 / 30 / 30) reached val loss* **0.007186**.

Mainline: hidden **128**, **8** steps, **≈1.29 M** params; **128 / 8** traded a little pilot accuracy for cheaper per-injection inference.

BACKUP · B3

Baseline vs noise-trained, against the solver



On a held-out test case, both GNN variants track the solver; the **noise-trained model** stays slightly closer late in the rollout.

back to talk

BACKUP · B4

The hook in PIMPLE, and one injection

```
while (pimple.loop())
{
    solver.prePredictor();
    if (pimple.firstIter())
    {
        solver.injectPredictorFields();    // hook
    }
    solver.momentumPredictor();
    solver.pressureCorrector();
}
```

1. **Gather and pack fields**
   copy U and p per cell, subtract the mean pressure, read the inlet speed
2. **TorchScript forward**
   host to GPU, normalize, 8 message-passing rounds, decode the increment
3. **Blend and write back**
   blend into the running field, add the mean pressure back, correct boundaries
4. **Solve the time step**
   momentum predictor and pressure correction continue unchanged

Injected once at the **first outer PIMPLE iteration**; the injection itself is gather, forward, blend, write back, then the solver continues.

The forked solver, and the dt-scaled blend

1. **Forked solver**
   `incompressibleFluidIterReduce` derives from OpenFOAM's `incompressibleFluid` and adds the one hook; installs to the user bin, stock solver untouched
2. **Coupling library**
   `libIterReduceCoupling` loads the exported module with `torch::jit::load` and runs its forward pass
3. **Build**
   both via `wmake` against standalone libtorch 2.5.1 (cu121, ABI 1, cxx11)

```
// blendMode = dtScaled
α = clamp( dtOF / dtGNN, 0, αmax )

u ← (1 − α)·unow + α·upred
p ← (1 − α)·pnow + α·ppred
```

Manual mode uses a constant **α = guessBlendAlpha**.

A small forked solver plus a coupling library; the **dt-scaled blend** mixes the prediction in proportion to the time-step ratio.

back to talk

BACKUP · B6

Iteration results in detail

Linear-solver iteration change vs cold baseline, aggregated over the 30-case test split

Configuration	Total	Pressure	Cases improved	Per-case mean
`Matched transient`	+21.6%	+14.8%	1 / 30	+22.2%
`Fine-step transient`	+18.0%	+23.5%	0 / 30	+19.0%
`Steady rollout-30`	–11.2%	–15.2%	30 / 30	–11.1%
`Steady rollout-100`	–17.9%	–21.7%	30 / 30	–17.9%

Aggregate % sum iteration counts over all 30 cases (steady cold baseline **56,848** *total iterations). Spread: matched* **–0.2% to +57.2%** *(median +19.3%); steady best case* **–27.6%** *(r30) and* **–32.8%** *(r100). Pressure dominates the change in every panel.*

Transient regresses on nearly every case; steady initialization improves **all 30**, which is what makes the learned field a credible initializer.

BACKUP · B7

Under-reaching, in detail

1 **Receptive-field limit**

a fixed depth gives only a K-hop reach, so one pass cannot carry the global pressure mode; the same reason single-level smoothers stall and OpenFOAM solves pressure with algebraic multigrid

2 **Solver inconsistency**

a field close in RMSE can still violate the continuity and pressure constraints that dominate the next solve; repeated every step, that adds pressure-solver work

Share of one-step error power in the lowest-wavenumber band (spectral centroid k):

pressure p	velocity u_x	velocity u_Y
90%	**60%**	**25%**
centroid $k = 1.3$	centroid $k = 1.9$	centroid $k = 3.3$

Pilot depth study: **128 / 12** *had the lowest validation loss, but* **128 / 8** *was chosen for cheaper inference at every injection.*

BACKUP · B8

Full wall-clock cost sheet

Measured cost per case (cumulative over 30 cases in the thesis) · Table 4.7

	CPU work in the process · comparable		GPU + disk, solver idle · wall-clock	
Configuration	**Execution, cold → model**	**of which: field handling**	**Model load**	**GNN forward**
`Matched transient`	3.8 → 5.8 s	+1.11 s	1.07 s	2.17 s
`Fine-step transient`	12.1 → 16.6 s	+1.26 s	1.08 s	3.67 s
`Steady rollout-30`	0.69 → 0.65 s	*offline*	0.06 s	0.13 s
`Steady rollout-100`	0.69 → 0.61 s	*offline*	0.06 s	0.27 s

Transient load and forward run **in-loop** *while the solver sits idle; steady costs run* **once offline** *before the solve. Execution time rises* **+52%** *matched and* **+37%** *fine-step; the steady solve itself runs faster but the offline generation is* **not yet repaid end-to-end**.

Solver execution is **CPU time**; model load and GNN forward are **wall-clock waits** while the solver sits idle.